

SMART HOSTEL MANAGEMENT SYSTEM

Submitted by :

Name : Prajjwal Pandey

Registration no : 25BAI11515

Academic Year: 2026 - 2027

Introduction :

Hostel administration involves many routine activities such as maintaining student details, checking room availability, collecting fees, recording visitors, monitoring attendance, and resolving complaints. When these details are maintained in paper registers or separate spreadsheets, it becomes difficult to update records quickly and to obtain a complete view of hostel operations.

The Smart Hostel Management System was developed to bring these activities into one desktop application. The project uses Java Swing to provide a graphical interface and keeps every module inside the same application window. The user can move between modules through a sidebar without opening many separate windows.

The system has been designed as a Programming in Java project. It applies object-oriented programming, collections, exception handling, file handling, multithreading, interfaces, abstract classes, inheritance, polymorphism, and JDBC-ready database code. The current version works with editable records during a session and can export records to CSV files. Its structure also allows MySQL persistence to be added later.

Problem Statement :

In many hostels, student information, room allocation, fee payments, visitor details, attendance, and complaints are recorded manually. This approach creates several problems: records may be duplicated, room availability may not be updated immediately, pending fees can be missed, and it takes time to prepare reports for the warden or administration.

The problem addressed by this project is the lack of a single and easy-to-use system for managing hostel operations. The Smart Hostel Management System provides a desktop-based solution where the administrator can view and update important hostel information from one application. It aims to reduce repetitive manual work, improve visibility of records, and make routine tasks easier to track.

Functional Requirements :

ID	Functional Requirement
FR-01	The system shall provide an administrator login screen before entering the application.
FR-02	The system shall display a dashboard containing student, room, complaint, visitor, and fee-related summary information.
FR-03	The system shall allow users to add, edit, delete, search, and view student records.
FR-04	The system shall allow rooms to be added and managed with capacity, occupied beds, available beds, and status.
FR-05	The system shall support automatic room allocation when a room with a free bed is available.
FR-06	The system shall record fee payments, show payment status, and display a receipt preview.
FR-07	The system shall register complaints, assign them to teams, and track their progress.
FR-08	The system shall record visitor entry and exit information and generate a visitor pass preview.
FR-09	The system shall record attendance status along with entry time and exit time.
FR-10	The system shall manage mess plans, outpass requests, inventory, notices, and staff records.
FR-11	The system shall support table search, validation, row editing, and deletion confirmation.
FR-12	The system shall export current table data to CSV files.
FR-13	The system shall provide settings for password, database connection, backup/restore controls, and syllabus concept examples.

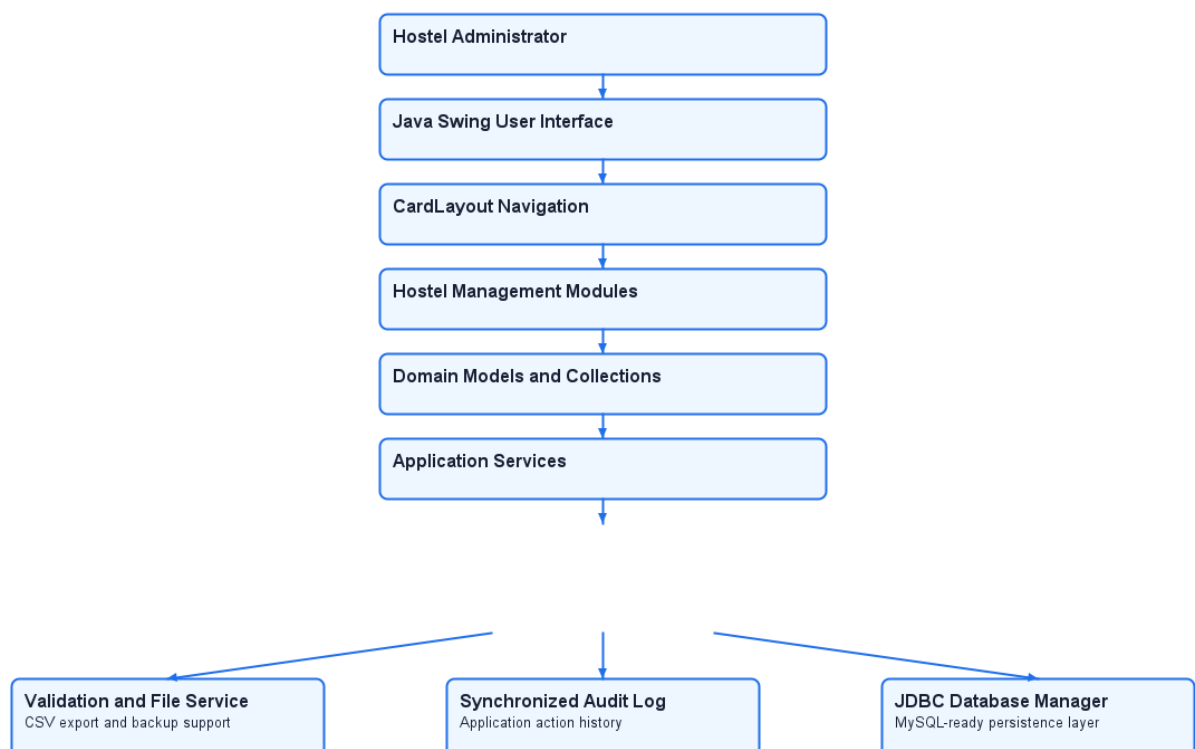
Non-functional Requirements :

Category	Requirement
Usability	The interface should be simple enough for hostel staff to use without technical training. It should use clear buttons, labels, tables, and visible scrollbars.
Performance	Common operations such as searching and editing in-memory records should respond immediately for normal hostel-scale data.
Reliability	The system should validate required fields, confirm deletions, and handle file or database errors without closing unexpectedly.
Maintainability	The code should use separate domain and utility classes so that new modules or database support can be added later.
Security	Access begins with a login screen. A production version should store passwords securely and restrict administrative settings to authorised users.
Portability	The application should run on Windows systems that have a compatible Java Development Kit installed.
Logging	Important actions can be written to a synchronized audit log for later review.

System Architecture :

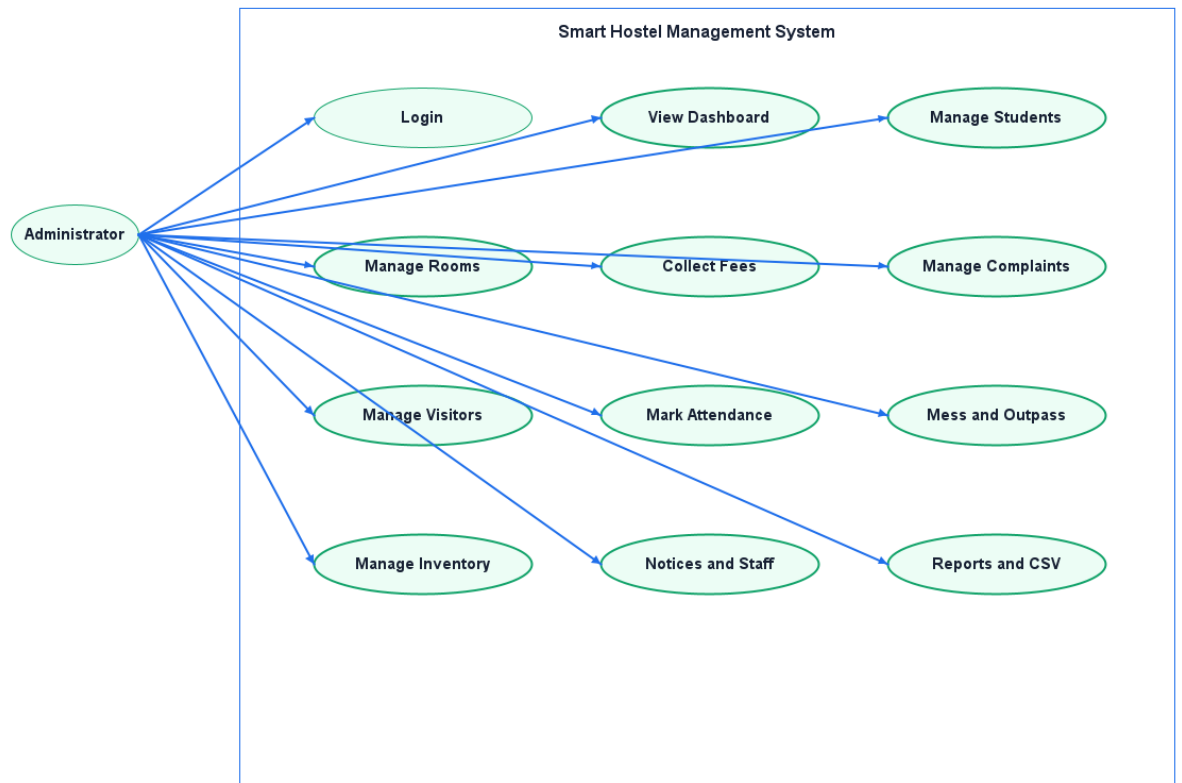
The application follows a simple layered structure suitable for a standalone Java Swing program.

1. Presentation layer : Java Swing components create the login screen, sidebar, dashboard, tables, forms, dialogs, and charts.
2. Navigation layer : `CardLayout` displays one module at a time inside the main application window.
3. Business/data layer : Domain classes such as `Student`, `Room`, `Payment`, `Complaint`, and `Visitor` represent hostel entities. Collections and table models maintain editable records during the session.
4. Service layer : File handling exports CSV records, the audit log records actions, and helper classes perform validation and room allocation.
5. Persistence layer : `DatabaseManager` contains JDBC methods for connection, query, update, and close operations. MySQL can be connected later by entering valid settings and adding the JDBC driver.

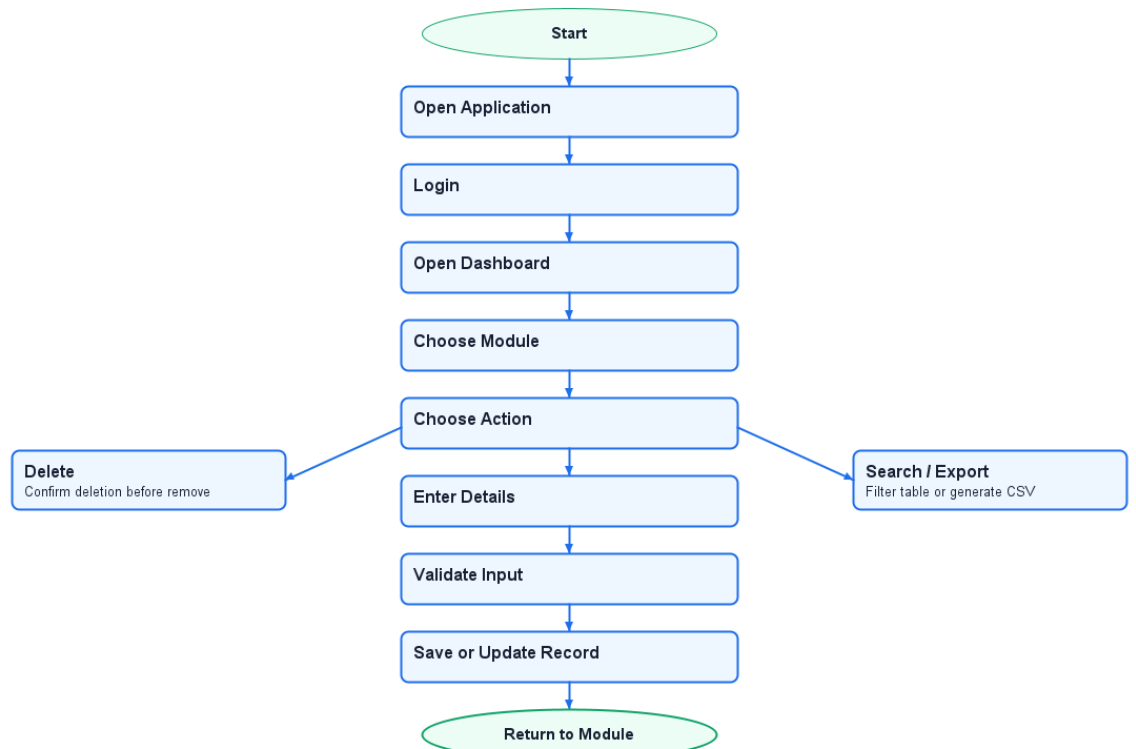


Design Diagrams :

1. Use Case Diagram

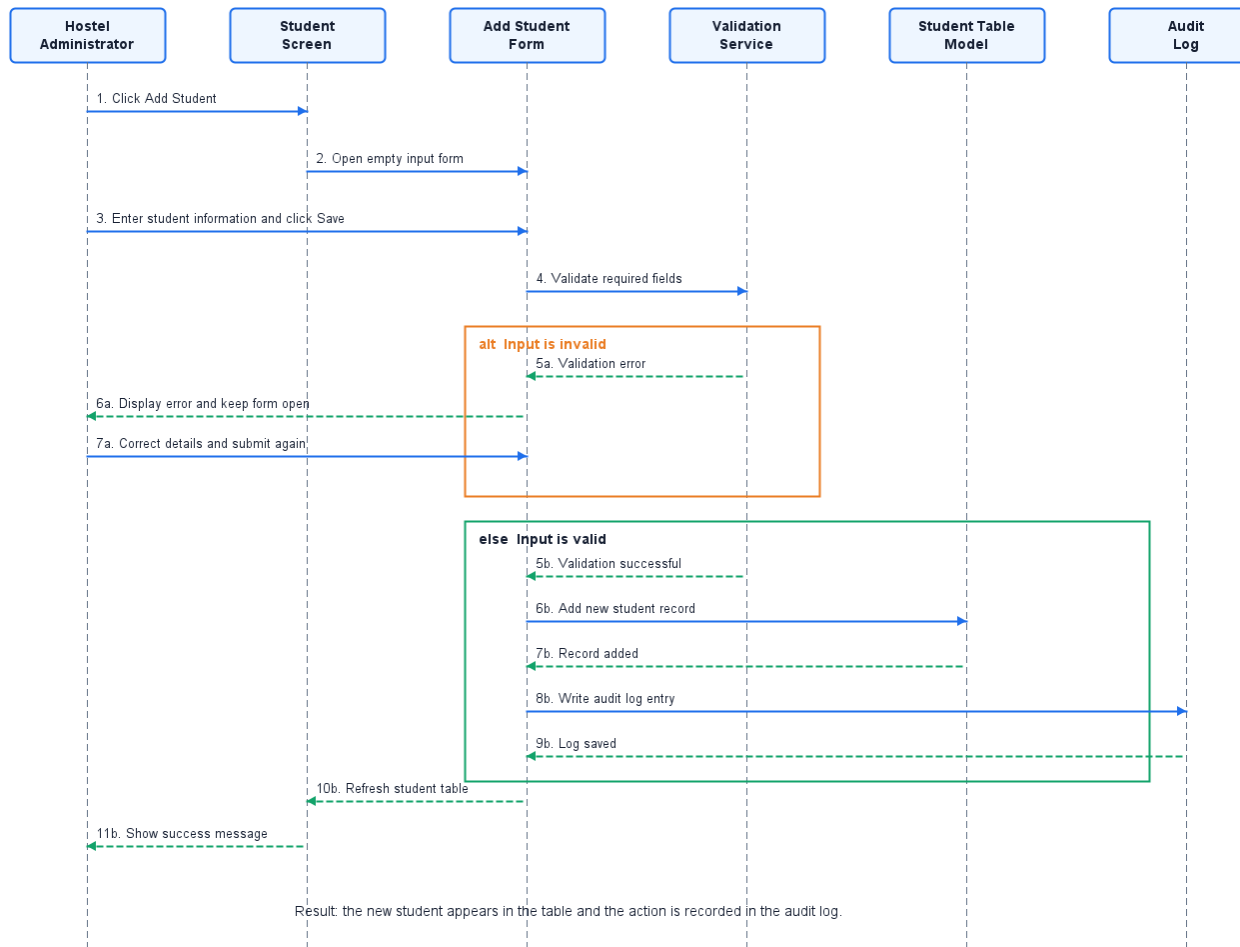


2. Workflow Diagram

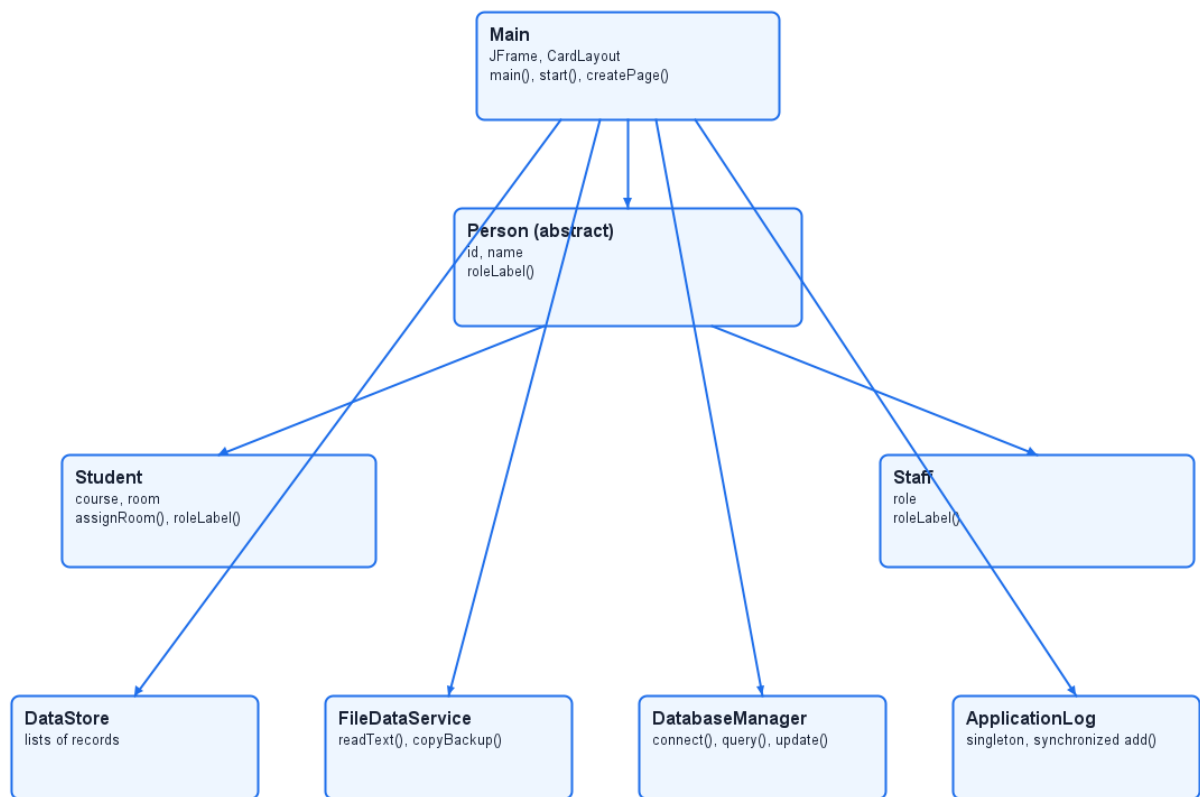


3. Sequence Diagram

Sequence Diagram - Add Student

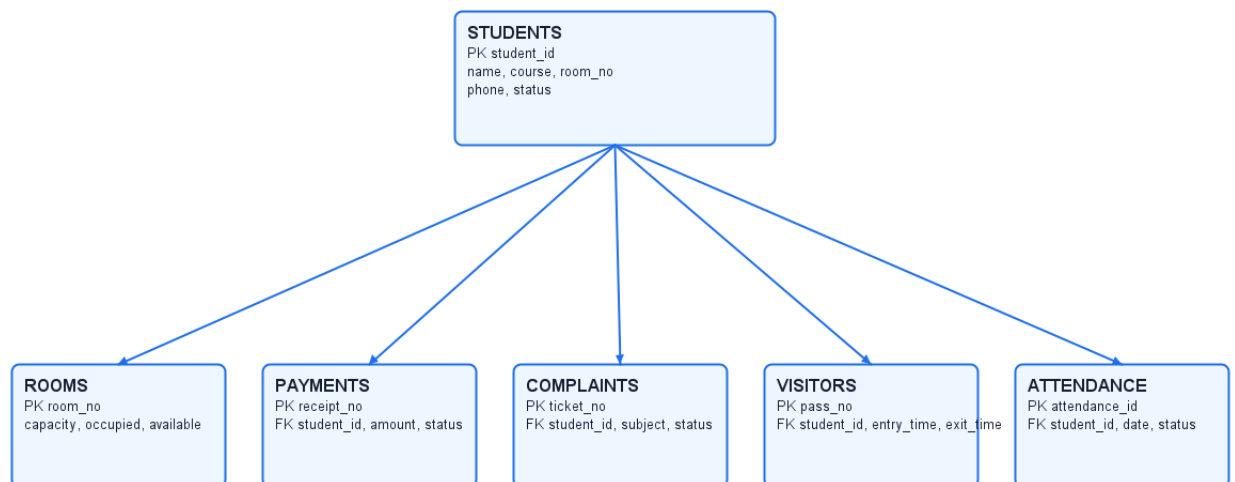


4. Class / Component Diagram



5. ER Diagram / Storage Design

ER Diagram - Proposed MySQL Database



Design Decisions and Rationale :

The project was developed as a single Java source file because the requirement was to run the complete application through `Main.java`. To keep the source understandable, related functionality was placed in nested classes. This allowed the application to demonstrate object-oriented design while still satisfying the one-file requirement.

Java Swing was selected because it is included with the JDK and is suitable for desktop applications. `CardLayout` was used so that all modules open in one main window instead of creating many separate frames. This gives the application a more consistent workflow.

`JTable` and `DefaultTableModel` were used for data presentation because hostel records are naturally tabular. `TableRowSorter` provides search functionality without requiring an external library. `JScrollPane` was added for the sidebar, dashboard, and tables so that the interface remains usable even when the window is smaller.

The project uses in-memory collections for the current version so that it can run immediately without requiring a database setup. At the same time, JDBC methods were included in `DatabaseManager` to show how the application can be connected to MySQL in a future version.

Implementation Details :

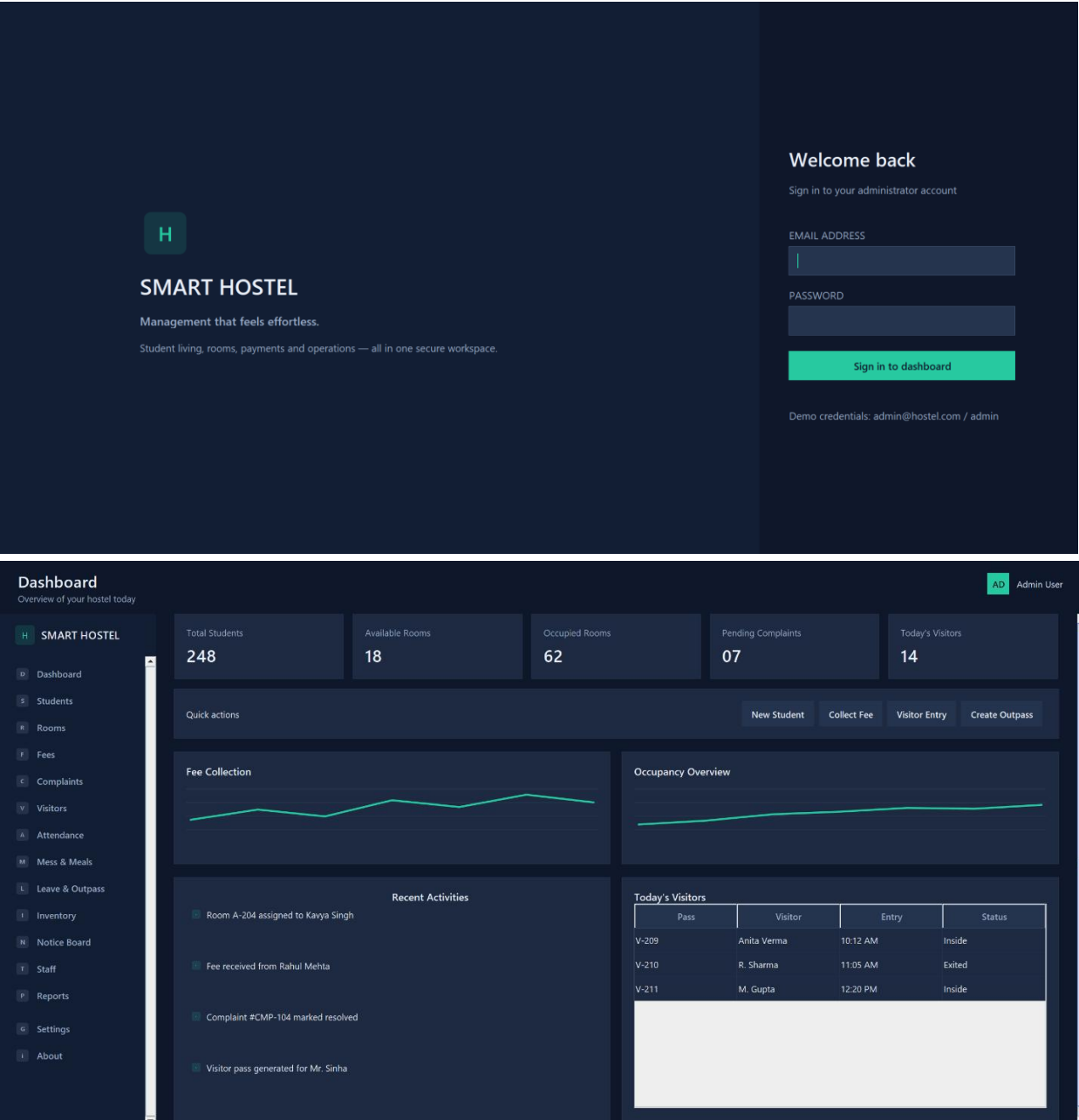
The project is implemented in Java using a single source file named `Main.java`. The main method starts the application on the Swing Event Dispatch Thread using `SwingUtilities.invokeLater()`.

The application opens with a login screen. After login, the main frame contains a fixed header, a scrollable sidebar, and a content area controlled by `CardLayout`. Each module is created as a panel and shown inside the same content area.

The system uses `DefaultTableModel` to keep module records editable during the session. The Add button opens a form where fields are validated. Edit Selected updates the chosen row, and Delete removes a row after user confirmation. The search box applies a filter to the table using `TableRowSorter`.

Several Java concepts were included deliberately for the course. `Person` is an abstract class, while `Student` and `Staff` extend it. `Identifiable` is an interface implemented by multiple entities. `MemoryRepository` demonstrates generic interfaces and collections. `ApplicationLog` demonstrates the singleton pattern and synchronization. `DashboardClock` extends `Thread`, while `FileDataService` demonstrates character and byte streams. `DatabaseManager` demonstrates JDBC connection and query structure.

Screenshots / Results :



Students

Manage students efficiently

ADAdmin User

SMART HOSTEL

Dashboard

Students

Rooms

Fees

Complaints

Visitors

Attendance

Mess & Meals

Leave & Outpass

Inventory

Notice Board

Staff

Reports

Settings

About

Search students...

Load Sample Students

View Profile

Export CSV

+ Add Student

Student ID	Name	Course	Room	Phone	Status
ST-1001	Aarav Sharma	B.Tech CSE	A-204	9876543210	Active
ST-1002	Kavya Singh	BBA	B-108	9876543211	Active
ST-1003	Rahul Mehta	B.Tech ECE	A-112	9876543212	Active
ST-1004	Neha Patel	MBA	C-305	9876543213	Fee Pending
ST-1005	Arjun Nair	BCA	B-211	9876543214	Active

Live records • Select a row to update or delete

Edit SelectedDelete

Fees

Manage fees efficiently

ADAdmin User

SMART HOSTEL

Dashboard

Students

Rooms

Fees

Complaints

Visitors

Attendance

Mess & Meals

Leave & Outpass

Inventory

Notice Board

Staff

Reports

Settings

About

Search fees...

Generate Receipt

Export CSV

+ Collect Fee

Receipt	Student	Amount	Payment Mode	Date	Status
REC-0522	Aarav Sharma	₹ 18,000	UPI	2026-09-05	Paid
REC-0523	Kavya Singh	₹ 18,500	Card	2026-09-04	Paid
REC-0524	Rahul Mehta	₹ 18,000	Cash	2026-09-03	Paid
REC-0525	Neha Patel	₹ 9,000	—	2026-09-01	Pending
REC-0526	Arjun Nair	₹ 18,000	Bank Transfer	2026-08-30	Paid

Live records • Select a row to update or delete

Edit SelectedDelete

Students

Manage students efficiently

ADAdmin User

SMART HOSTEL

Dashboard

Students

Rooms

Fees

Complaints

Visitors

Attendance

Mess & Meals

Leave & Outpass

Inventory

Notice Board

Staff

Search students...

Load Sample Students

View Profile

Export CSV

+ Add Student

Student ID	Name	Course	Room	Phone	Status
ST-1001	Aa		204	9876543210	Active
ST-1002	Ka		108	9876543211	Active
ST-1003	Ra		112	9876543212	Active
ST-1004	Ne		305	9876543213	Fee Pending
ST-1005	Ar		211	9876543214	Active

Live records • Select a row to update or delete

Edit SelectedDelete

Add Student

STUDENT ID

STU-106

NAME

Krishna

COURSE

CSE

ROOM

Enter Room

PHONE

Enter Phone

STATUS

Active

OKCancel

Testing Approach :

Testing was performed manually because the project is a GUI-based desktop application. Each module was opened from the sidebar and its main operations were checked using valid and invalid input.

Test Case	Expected Result
Login with non-empty credentials	Dashboard opens successfully.
Add a student with complete data	A new row appears in the student table.
Edit selected student	The selected record changes in the table.
Delete selected student	Confirmation is requested and the row is removed after approval.
Search student by name	Only matching rows are displayed.
Export CSV	A CSV file containing the current table data is created.
Auto allocate room	A room with an available bed is allocated, if available.
Resize the window	Sidebar, dashboard, and tables remain scrollable.
Open syllabus concept demo	Java concept summary is displayed without stopping the application.

Challenges Faced :

One challenge was building a large hostel-management interface while keeping the project in one Java file. This was addressed by using nested classes and keeping related code grouped together.

Another challenge was making the user interface easy to navigate even with many modules. A scrollable sidebar and `CardLayout` were used to keep the application organized. The dashboard and tables were also placed inside scroll panes so that content remains visible on smaller screens.

Maintaining correct data in each module was also important. At first, generic sample values could appear under unrelated table columns. This was corrected by defining separate sample data for students, rooms, payments, complaints, visitors, attendance, and other modules.

Finally, permanent database storage was not configured in the current version because it would require a running MySQL server and driver setup. The project therefore uses in-memory data for demonstration while preserving JDBC methods for future integration.

Learnings and Key Takeaways :

This project provided practical experience in creating a Java desktop application rather than only writing small console programs. It showed how a real application can be divided into user-interface code, domain classes, data collections, utility services, and database access code.

The project improved understanding of OOP concepts such as inheritance, abstraction, interfaces, encapsulation, and runtime polymorphism. It also demonstrated the importance of validation and exception handling in user-facing applications. Working with `JTable`, `CardLayout`, `JScrollPane`, and dialogs improved understanding of Swing event-driven programming.

The project also highlighted that data handling and UI design are connected. Clear labels, correct table columns, scroll support, and useful error messages make a significant difference to the usability of an application.

Future Enhancements :

- Connect all modules to a MySQL database using the existing JDBC structure.
- Add secure password hashing and role-based login for warden, staff, and security users.
- Generate printable PDF receipts and reports.
- Add email or SMS notifications for pending fees, complaints, and approved out passes.
- Add QR-code visitor passes and biometric/RFID attendance support.
- Implement database backup scheduling and restore verification.
- Add student document upload and guardian contact management.
- Add room-wise analytics and monthly fee collection charts based on real database data.

References :

1. Herbert Schildt, **Java: The Complete Reference**, 11th Edition, Oracle Press, 2018.
2. Deitel and Deitel, **Java How to Program: Early Objects**, 10th Edition, Pearson, 2015.
3. Oracle Java Documentation, Java SE API Documentation. Available at: <https://docs.oracle.com/en/java/>
4. Oracle Java Tutorials, Swing and JDBC documentation. Available at: <https://docs.oracle.com/javase/tutorial/>
5. Course syllabus: CSE2006 - Programming in Java, VIT.